

960121

The Product Catalog

Session 02



Session 2

- **The Product Catalog**, we are moving from static templates to understanding how data flows. As full-stack developers, we don't just "write code"; we design interactions.
- Before we touch a single line of JavaScript, we need to map out the **Business Logic**—the "**why**" and "**how**" behind the user experience.

UML

- **U**nified **M**odeling **L**anguage (UML) is a standardized, general-purpose visual modeling language used to specify, visualize, construct, and document software systems and business processes.
- It provides a blueprint for software engineering, using diagrams to represent structural (e.g., class diagrams) and behavioral (e.g., use case diagrams)

UML

- **Structure vs. Behavior:** UML diagrams are divided into two main categories:
 - **Structural Diagrams:** Define the static structure of a system (e.g., Class Diagrams, Object Diagrams, Component Diagrams).
 - **Behavioral Diagrams:** Define the dynamic behavior of a system (e.g., Use Case Diagrams, Sequence Diagrams, Activity Diagrams).

Use Case Diagram

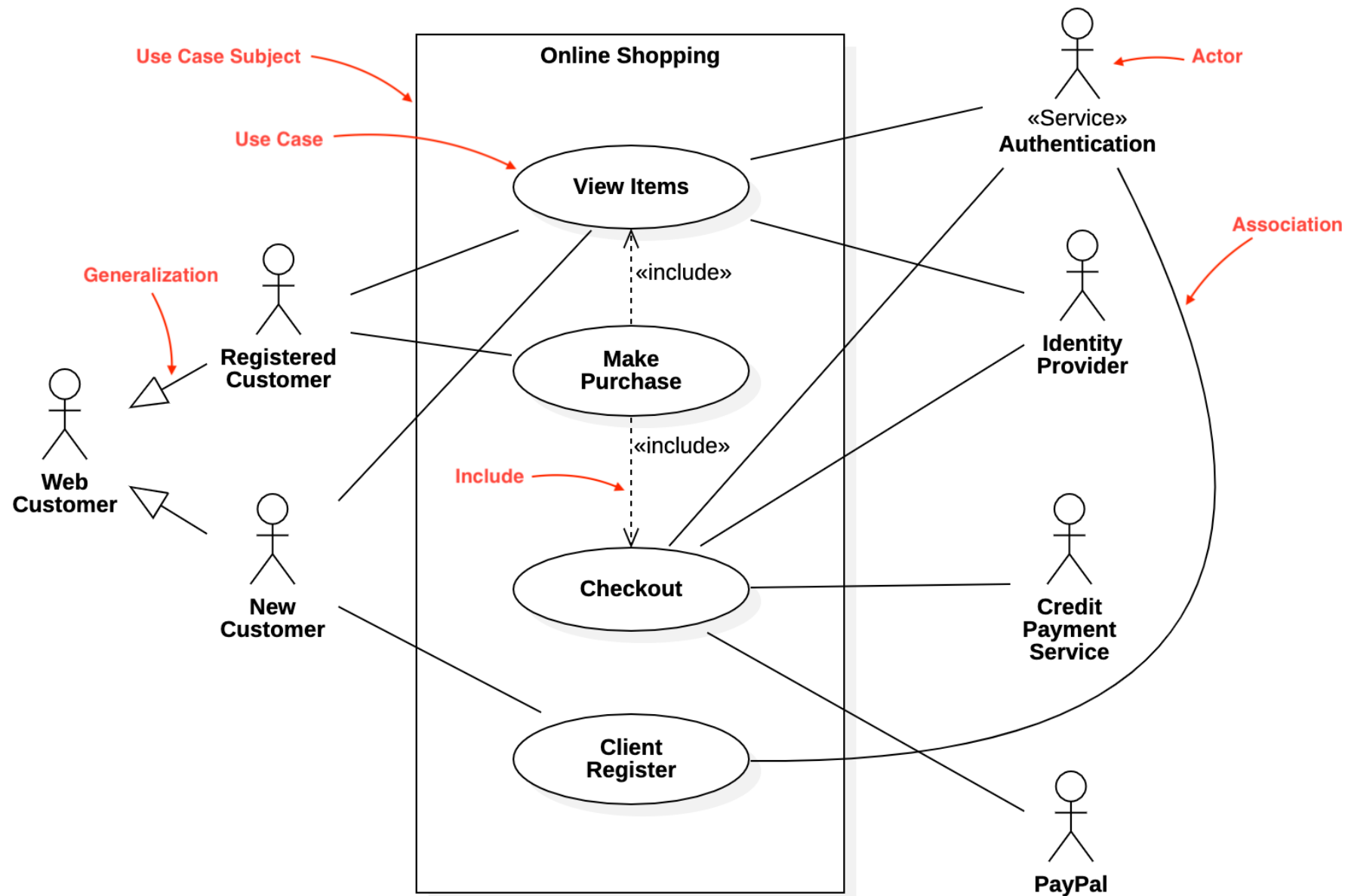
Represent:

- Scenarios in which your system or application interacts with people, organizations, or external systems.
- Goals that your system or application helps those entities (known as actors) achieve.
- The scope of your system.

Use Case Diagram

Components:

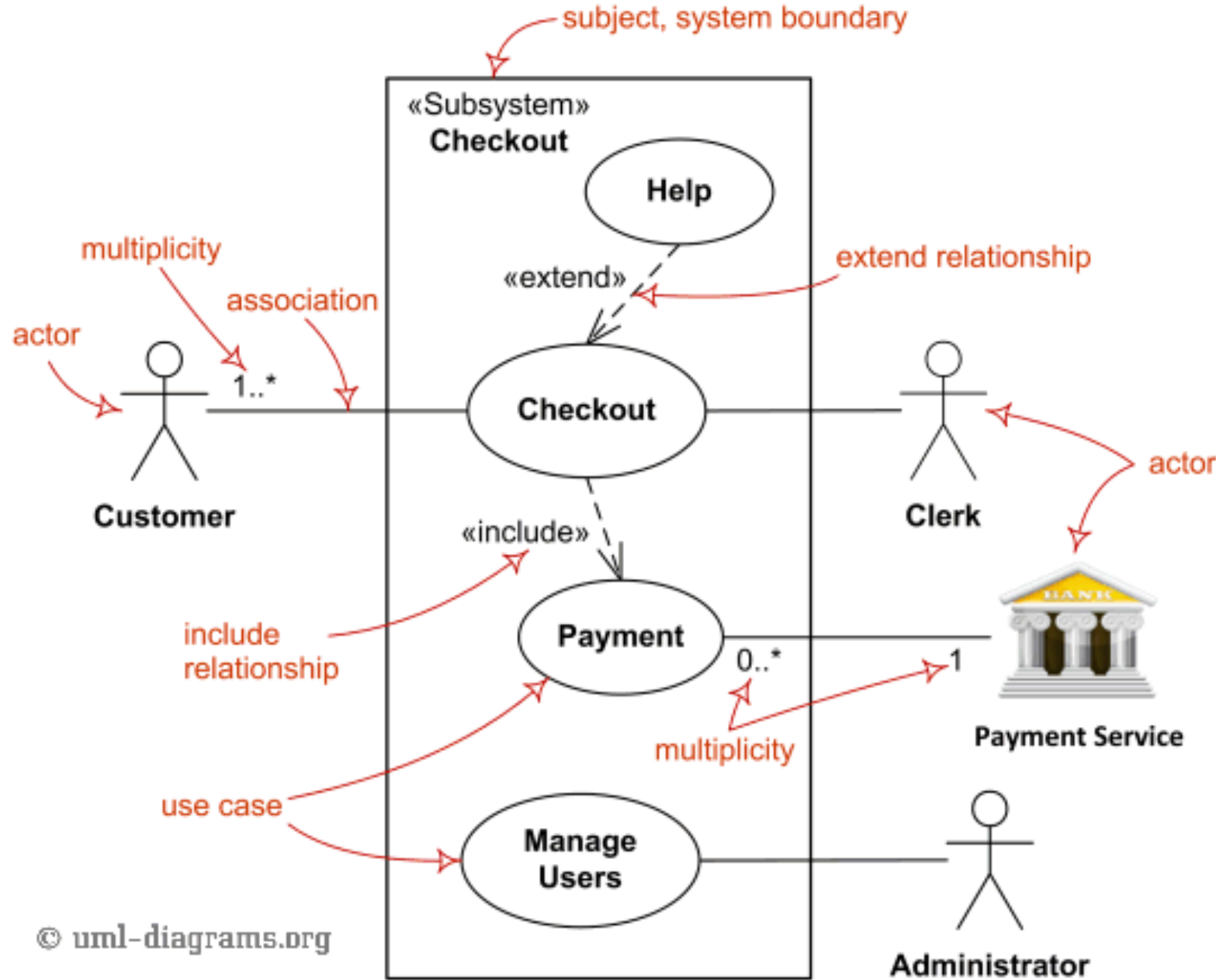
- **Actors:** The users that interact with a system. An actor can be a person, an organization, or an outside system that interacts with your application or system. They must be external objects that produce or consume data.
- **System:** A specific sequence of actions and interactions between actors and the system. A system may also be referred to as a scenario.
- **Goals:** The end result of most use cases. A successful diagram should describe the activities and variants used to reach the goal.



Use Case Diagram

- **Use cases:** Horizontally shaped ovals that represent the different uses that a user might have.
- **Actors:** Stick figures that represent the people actually employing the use cases.
- **System boundary boxes:** A box that sets a system scope to use cases. All use cases outside the box would be considered outside the scope of that system.

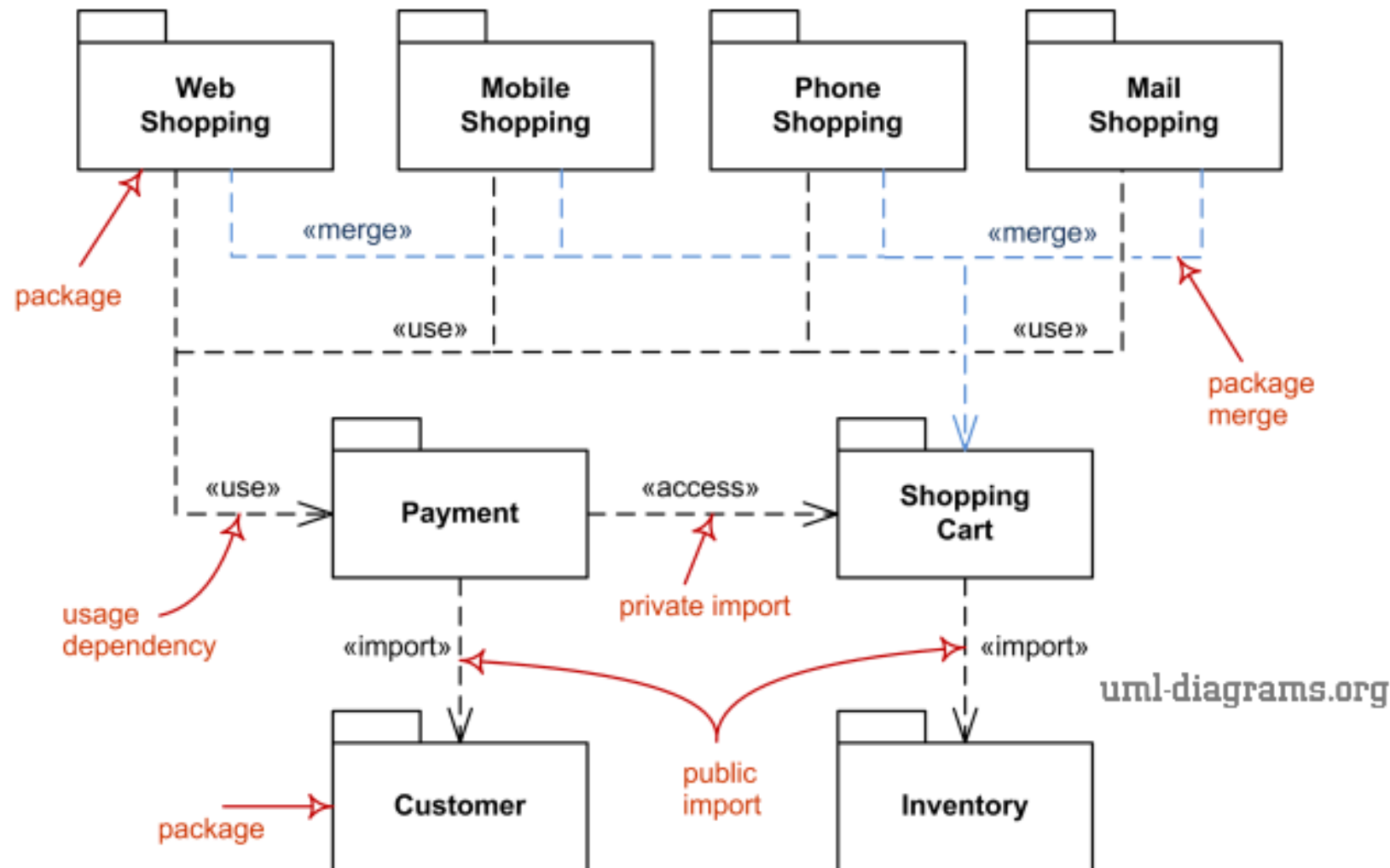
Use Case Diagram



- **Associations:** A line between actors and use cases. In complex diagrams, it is important to know which actors are associated with which use cases.

Use Case Diagram

- **Packages:** A UML shape that allows you to put different elements into groups. Just as with component diagrams, these groupings are represented as file folders.



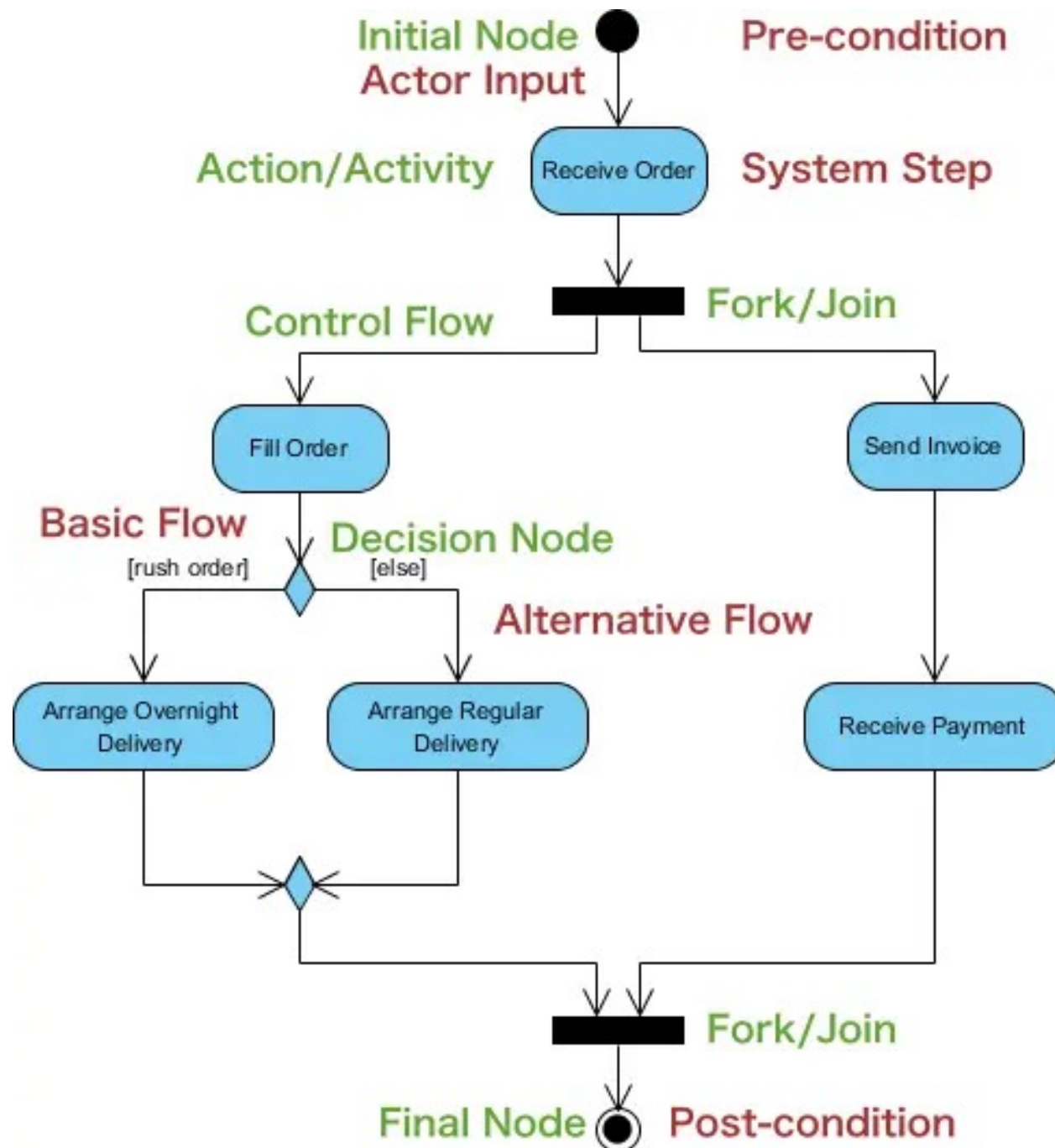
Activity Diagram

- A UML activity diagram is a **behavioral diagram** that visualizes the **flow of control**, **decisions**, and **parallel actions** within a system or business process.
- It illustrates the step-by-step sequence of actions—from start to finish—making complex workflows understandable for both technical and non-technical stakeholders.

Activity Diagram

Activity Diagram vs. Flowchart

- While similar, activity diagrams are superior in representing **parallel processing** (concurrent behavior) and complex interactions within a system, whereas flowcharts are usually linear.
 - Modeling business processes or workflows.
 - Describing the logic of algorithms or software systems.
 - Modeling the **behavior of a use case** (e.g., student enrollment, purchasing a product).

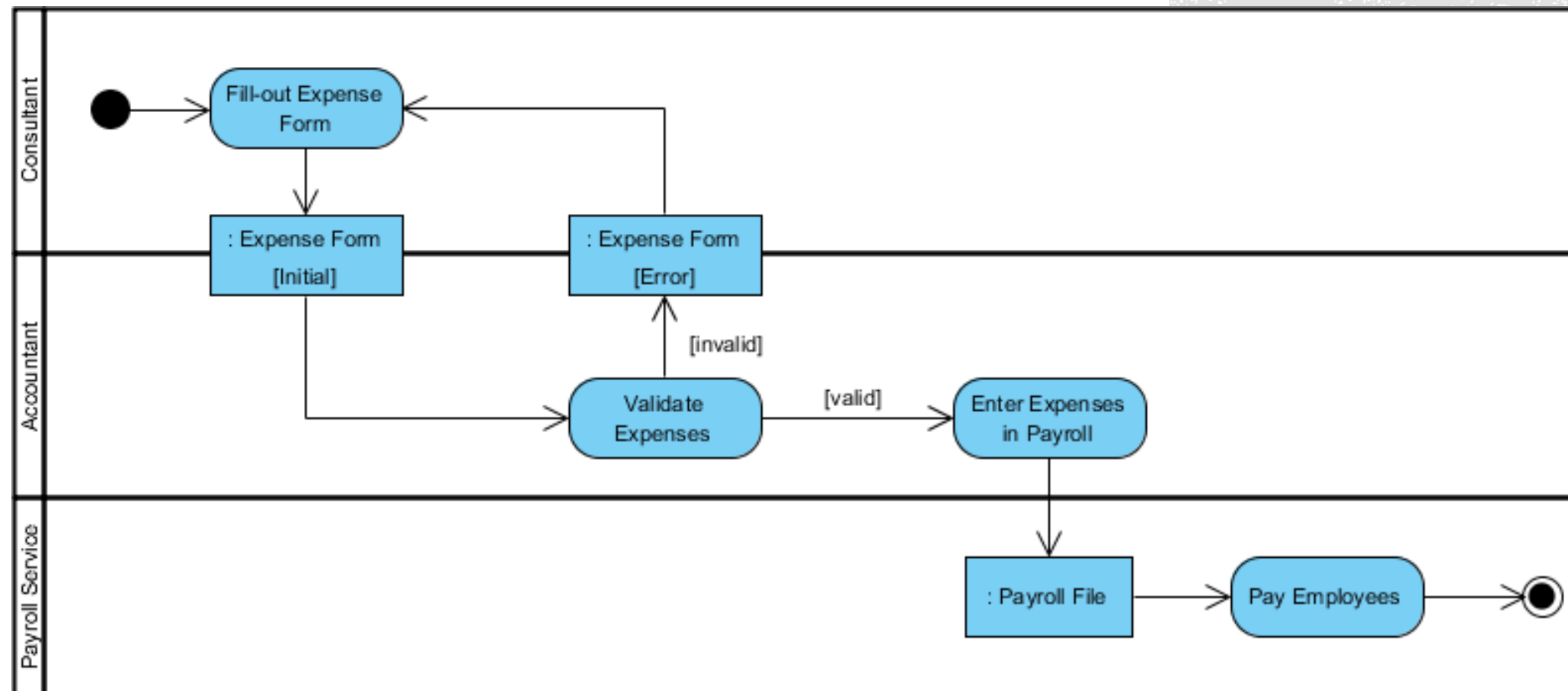


Activity Diagram

- **Initial Node:** the start.
- **Action/Activity:** representing a task to be performed.
- **Control Flow:** Arrow showing the sequence of execution.
- **Decision Node:** Diamond for conditional branching.
- **Fork/Join:** Thick solid line to split (fork) or combine (join) parallel activities.
- **Final Node:** marking the end of the process.

Activity Diagram

- **Swimlanes:** Vertical/horizontal columns used to divide activities based on the actor or department responsible.

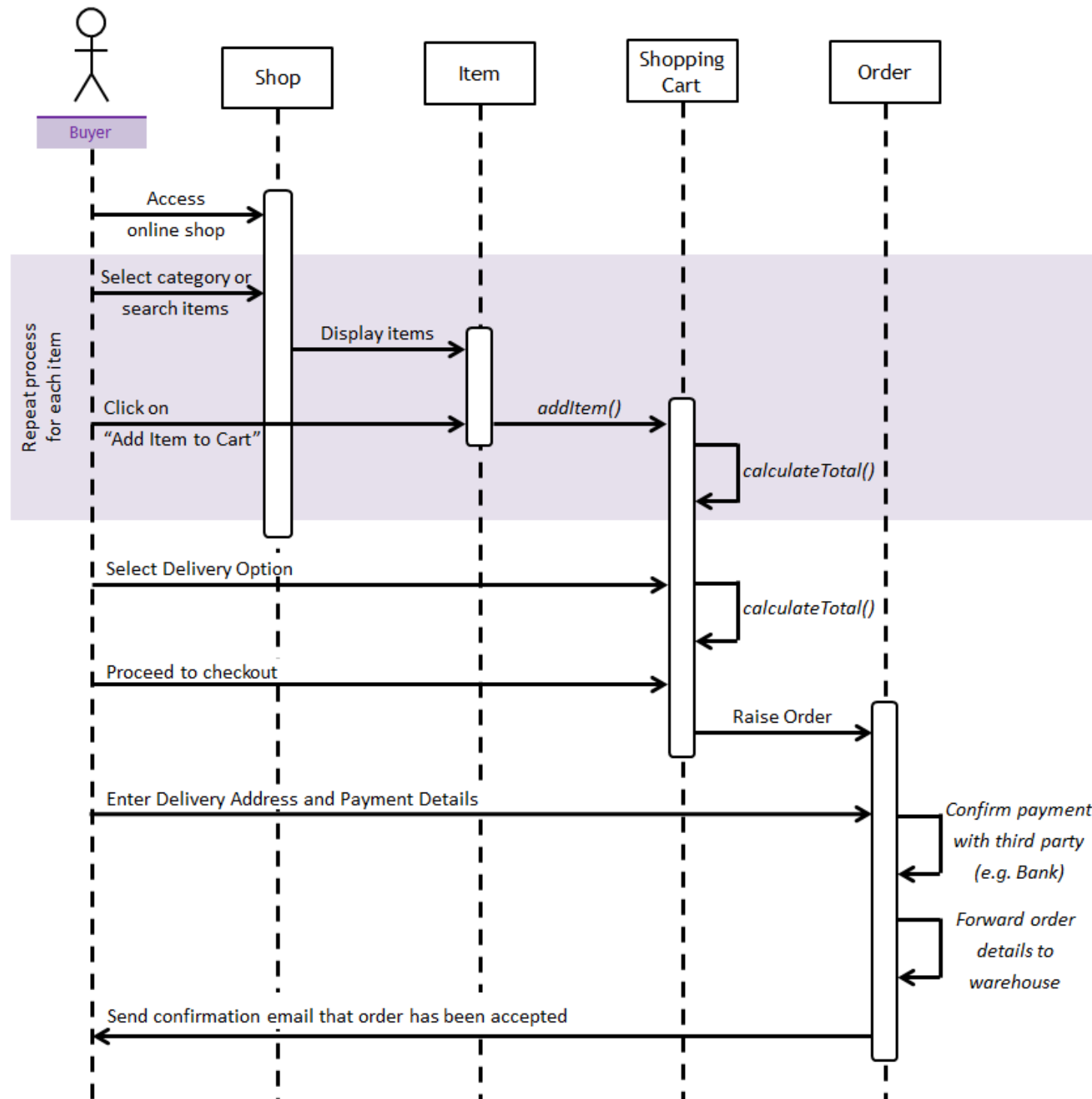


Sequence Diagram

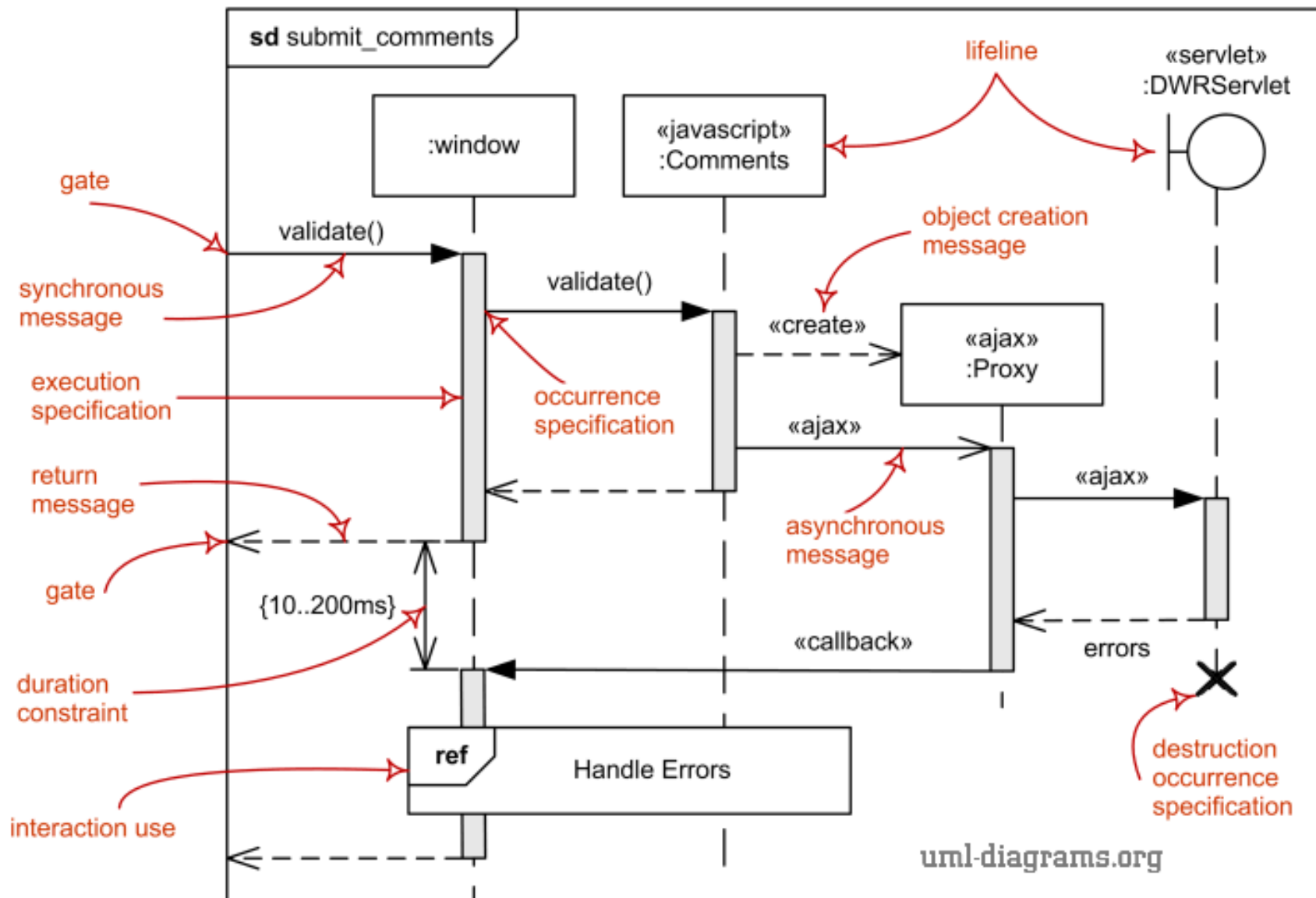
- A UML Sequence Diagram is an **interaction diagram** detailing how objects operate and communicate over time within a system, ordered sequentially.
- It visualizes the flow of messages between actors and components, reading top-to-bottom, to map out scenarios, user interactions, or API calls, essential for design and documentation.

Sequence Diagram

- **Actors/Objects**
(Participants): Represented at the top by rectangles, showing who or what is interacting (e.g., User, System, Database).
- **Lifelines**: Vertical dashed lines extending from objects, representing their existence over time.



Sequence Diagram



- **Messages:** Horizontal arrows connecting lifelines, indicating communication (e.g., synchronous, asynchronous, or return messages).
- **Activation Boxes:** Thin rectangles on lifelines indicating that an object is actively performing an operation.
- **Fragments:** Frames used to represent complex logic such as alt (if/else), opt (optional), or loop.

Sequence Diagram

Common Use Cases

- **Documenting Use Cases:** Visualizing detailed step-by-step logic.
- **API/Service Design:** Mapping interactions between microservices.
- **System Design:** Clarifying how objects collaborate to perform functions.

Best Practices

- **Keep it Simple:** Avoid excessive detail that clutters the diagram.
- **Focus on Time:** Ensure the top-to-bottom order accurately represents the process flow.



Referance

- **The Unified Modeling Language**
(<https://www.uml-diagrams.org/>)
- **UML Use Case Diagram Tutorial**
(<https://www.lucidchart.com/pages/tutorial/uml-use-case-diagram>)
- **Use Case Diagram**
(<https://docs.staruml.io/working-with-uml-diagrams/use-case-diagram>)
- **StarUML documentation**
(<https://docs.staruml.io/>)

Class Activity

(20 min.) Warmup: From your E-commerce website

- **(5 min.):** Draw **Use Case Diagram** for general user and customer/member
- **(5 min.):** Draw **Activity Diagram** for customer/member registration
- **(5 min.):** Draw **Sequence Diagram** for customer/member login
- **(5 min.):** Peer Evaluation

Critical Thinking Question

Designing the Catalog

1. (5 min.) **The Scenario:**

- **The Challenge:** Your e-commerce template currently has "hard-coded" products in HTML. If we have 1,000 products, we can't hand-write 1,000 HTML blocks.
- **Goal:** We need to fetch this data dynamically using a GET request so the UI updates automatically when the data changes.
- Explore and locate products block/grid in your HTML

Designing the Catalog

2. (10 min.) UML:

- Draw **Sequence Diagram** for getting product data and display on website.
- **Label the arrows:**
 - `requestProducts()`,
 - `fetch()`,
 - `return JSON`, and
 - `renderProducts()`.
- **Actors/Object:** ???

Designing the Catalog

3. (5 min.) **Peer Evaluation:**

- Students swap drawings.
- **Critical Thinking Question:**
 - Did your friend account for a loading state?
 - What happens if cannot find/load data?
 - What happens while the data is traveling?

Designing the Catalog

4. (5 min.) GenAI Prompt Engineering:

- Students use UML logic to craft a structured prompt.

Prompt: (generate JSON file)

“Act as a Full Stack Developer. Generate a JSON file with 20 products structured according to the given product item HTML block.”

- Copy HTML block and paste into your GenAI.
- Save generated JSON file in your web project folder (./data/json)

Designing the Catalog

4. (5 min.) **GenAI Prompt Engineering:**

- Practice writing prompt on paper or Notepad first.

Designing the Catalog

5. (5 min.) **Integration:**

- Open your one-page template in VS Code.
- Replace the static `<div class="product-item">` blocks with a single container
- Paste the generated logic into your `<script></script>` or `script.js`.

Designing the Catalog

6. (10 min.) Debugging & Testing:

- Open your web on localhost via VS Code (Live Server).
- If found error, use Browser DevTools to debug code.
- Test “**Fail**” case: purposely rename your JSON file to see if their "Error Catch" logic from the UML diagram actually works in the browser console.
- Use Browser DevTools to see the GET request in real-time.

Designing the Catalog

7. (5 min.) Learn from GenAI:

Prompt:

“Modify the given JavaScript code to match my sequence diagram, which includes the `requestProducts()`, `fetch(path-to-json-file)`, and `renderUI()` functions. Add comments to explain in more detail how this code works and how the data were flow.”

- Copy your final (worked) JavaScript code into your GenAI.

Designing the Catalog

7. (5 min.) Learn from GenAI:

Prompt:

“Explain the concept behind the `fetch()` function, including what are AJAX and JSON (format and comparison to array), and how they work together”

- Copy your final (worked) JavaScript code into your GenAI.

Designing the Catalog

8. (5 min.) Version Control:

- Terminal Command:
 - `git add`
 - `git commit -m "feat: implement dynamic product fetching via GET"`
 - `git push.`
- VS Code:
- GitHub Desktop:

Designing the Catalog

8. Version Control:

- Reflection: Why do we use "**feat**" in the commit message?
- The use of "**feat**" (short for feature) comes from a widely adopted industry standard called **Conventional Commits**.
 - Automated Versioning
 - Meaningful Changelogs
 - Faster Code Reviews

Designing the Catalog

8. Version Control: A proper conventional commit follows this pattern:

`type(optional-scope) : description`

- `feat`: A new feature for the user.
- `fix`: A bug fix.
- `docs`: Documentation changes only.
- `style`: Changes that do not affect the meaning of the code (white-space, formatting, missing semi-colons).
- `refactor`: A code change that neither fixes a bug nor adds a feature.
- `test`: Adding missing tests or correcting existing tests.

Key Takeaway

- **AI is the "Assistant," but the UML is the "Manager."**
- If the UML diagram is wrong, the AI will confidently write "perfect" code for the wrong solution.
- Critical thinking is the filter that ensures the AI's output aligns with the Business Logic.